

DRA-драйвер на Go: два gRPC-метода для управления GPU

Вся логика управления железом
— внутри драйвера

Станислав Смирнов

Go-разработчик, Deckhouse Kubernetes Platform
Направление ML/AI-инфраструктуры

«Флант» — инженерная компания; делаем Deckhouse,
платформу на базе Kubernetes



**GPU вам выдаёт
не Kubernetes**

GPU вам выдаёт не Kubernetes

- Не планировщик,
не API-сервер — а программа
на **Go** в каждом узле

GPU вам выдаёт не Kubernetes

- Не планировщик,
не API-сервер — а программа
на **Go** в каждом узле
- Два метода — **Prepare**
и **Unprepare**. Внутри — cgo,
NVML, нарезка карты

GPU вам выдаёт не Kubernetes

- Не планировщик,
не API-сервер — а программа
на **Go** в каждом узле
- Два метода — **Prepare**
и **Unprepare**. Внутри — cgo,
NVML, нарезка карты
- От кода между ними зависит:
парк GPU работает на **30 %** или на **80 %**

8 узлов × 8 H100 = 64 GPU



Узел из восьми H100 — небольшой автопарк.
Он зарабатывает, только когда реально считает.

Один парк GPU – три разных мира

Один парк GPU — три разных мира

1. **Обучение**
Честные целые карты,
пачкой и сразу

Один парк GPU — три разных мира

1. **Обучение**
Честные целые карты,
пачкой и сразу
2. **Продовый инференс**
Часто не нужна вся
H100 — хватит доли

Один парк GPU — три разных мира

1. **Обучение**
Честные целые карты,
пачкой и сразу
2. **Продовый инференс**
Часто не нужна вся
H100 — хватит доли
3. **Мелочь: CI, профилировки, batch**
Четвертинка карты на пять минут

Device plugin: язык один

resources:

limits:

nvidia.com/gpu: 1

обучение, инференс, CI – все говорят одно

Число — слишком бедный язык

Число — слишком бедный язык

- Нельзя выразить долю карты или «30 % памяти»

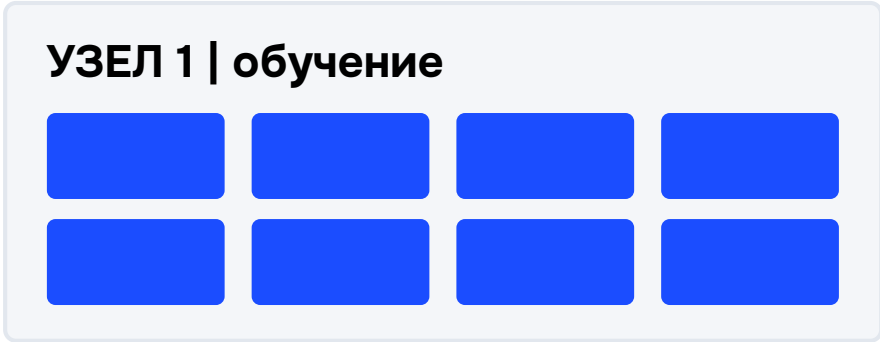
Число — слишком бедный язык

- **Нельзя выразить** долю карты или «30 % памяти»
- **Нет атрибутов:**
ни модели, ни памяти,
ни профиля, ни топологии

Число — слишком бедный язык

- **Нельзя выразить** долю карты или «30 % памяти»
- **Нет атрибутов:**
ни модели, ни памяти,
ни профиля, ни топологии
- **Планировщик видит**
лишь «nvidia.com/gpu: 8»
на уровне kubelet

Получаем простой ресурсов



 Загружена  Недогружена  Простаивает  Подождёт

Получаем простой ресурсов

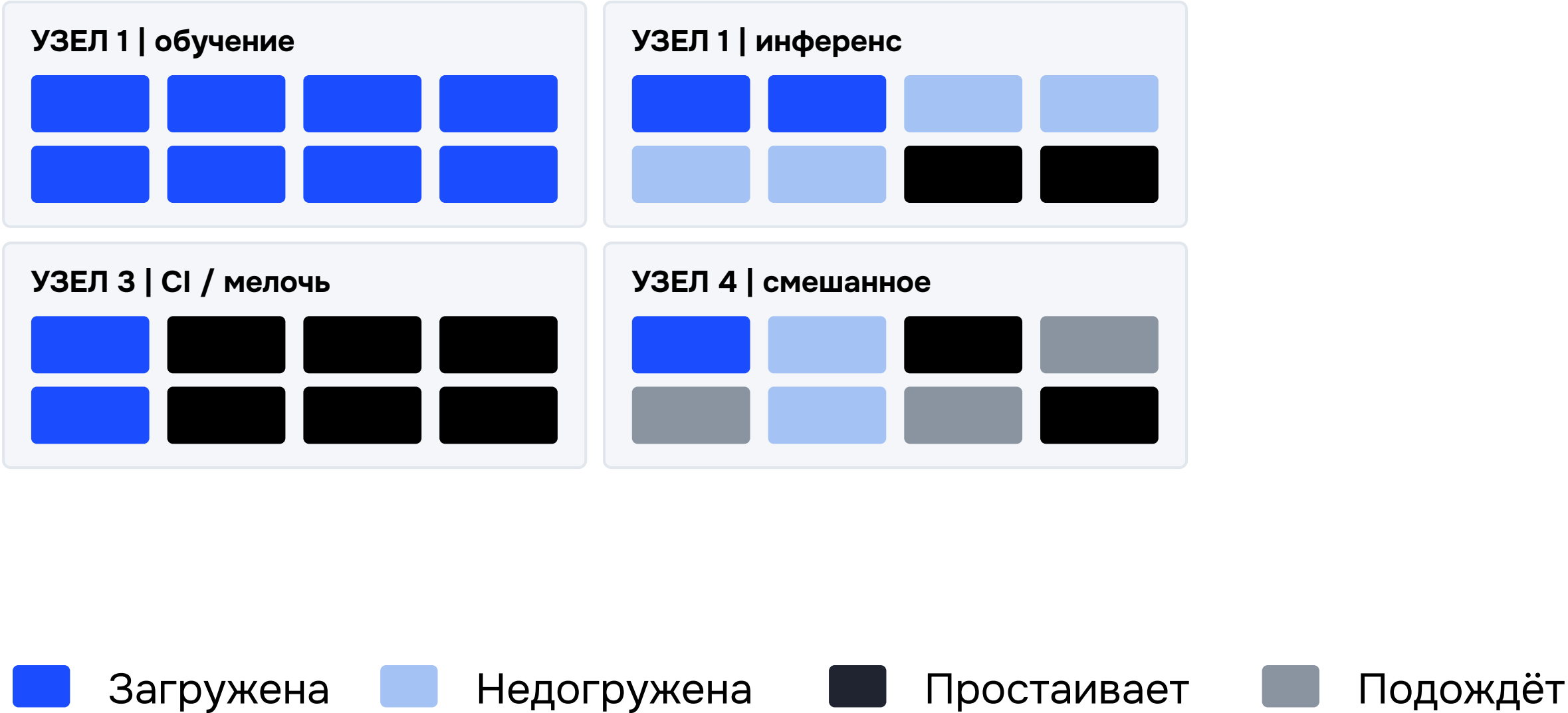


■ Загружена ■ Недогружена ■ Простаивает ■ Подождёт

Получаем простой ресурсов



Получаем простой ресурсов



Получаем простой ресурсов



Целую карту отдаём под задачу, которой хватило бы четверти.

32 карты «заняты» — а работает треть.

■ Загружена ■ Недогружена ■ Простаивает ■ Подождёт

Готовые решения
описывают GPU...
лейблами

Готовые решения описывают GPU... лейблами

- **Device plugin.**
Целое число `nvidia.com/gpu` –
ни модели, ни памяти, ни профиля

Готовые решения описывают GPU... лейблами

- **Device plugin.**
Целое число `nvidia.com/gpu` –
ни модели, ни памяти, ни профиля
- **NA Mi, Volcano.**
Навешивают на узлы лейблы
и аннотации – но это внешние метки,
не модель ресурса

Готовые решения описывают GPU... лейблами

- **Device plugin.**
Целое число nvidia.com/gpu –
ни модели, ни памяти, ни профиля
- **NA Mi, Volcano.**
Навешивают на узлы лейблы
и аннотации – но это внешние метки,
не модель ресурса
- **Разнородные нагрузки не описать.**
Инференсу, ML-обучению и CI нужны
разные атрибуты одной карты

**Поделиться карту
можно, но у каждого
способа своя цена**

**Поделить карту
можно, но у каждого
способа своя цена**

- **Аппаратный vGPU.**
До 7 виртуальных GPU, но жёстко завязан
на вендора и лицензии

**Поделить карту
можно, но у каждого
способа своя цена**

- **Аппаратный vGPU.**
До 7 виртуальных GPU, но жёстко завязан на вендора и лицензии
- **Квотирование.**
Делит, но проседает производительность

Поделить карту можно, но у каждого способа своя цена

- **Аппаратный vGPU.**
До 7 виртуальных GPU, но жёстко завязан на вендора и лицензии
- **Квотирование.**
Делит, но проседает производительность
- **Time-slicing.**
Говоришь, что карт не 8, а 80 —
и Kubernetes не контролирует загрузку

Поделить карту можно, но у каждого способа своя цена

- **Аппаратный vGPU.**
До 7 виртуальных GPU, но жёстко завязан на вендора и лицензии
- **Квотирование.**
Делит, но проседает производительность
- **Time-slicing.**
Говоришь, что карт не 8, а 80 —
и Kubernetes не контролирует загрузку
- **Ручное переключение MIG-режима.**
Делает администратор руками — ошибся,
и ресурс утёк

DRA – Dynamic Resource Allocation

Штатный механизм Kubernetes.
Stable и включён по умолчанию с 1.35.

DRA сделал с GPU то же,
что **StorageClass** — с диском.

Ресурс перестал быть числом и стал сущностью
с атрибутами и правилами выбора.

Как было: device plugin

Pod — что пишет пользователь

```
c
resources:
  limits:
    nvidia.com/gpu: 8
```



Механика device plugin:

kubelet · device plugin

публикует целое число

Узел

nvidia.com/gpu: 8 — всё или ничего

Пользователь пишет целое число, планировщик его вычитает — карта целиком

Как стало: меняется одна строка

Pod — что пишет пользователь

```
resources:  
  limits:  
    gpu.deckhouse.io/h100: 8
```



Поменялось только имя ресурса — extendedResourceName.
За кулисами — 4 объекта DRA, все из апстрима Kubernetes (resource.k8s.io):

ResourceSlice

инвентарь карт

DeviceClass

какие подходят

ResourceClaimTemplate

заявка

Pod

связь с заявкой

Переход простой: пользователь меняет одну строку — остальное «под капотом»

ResourceSlice – инвентарь (публикует драйвер)

kind: ResourceSlice

spec:

driver: gpu.deckhouse.io

devices:

- name: gpu-0

attributes: { model: {string: "H100" } }

capacity: { memory: {value: 80Gi} }

DeviceClass – правила выбора на CEL

```
kind: DeviceClass
metadata: { name: nvidia-h100-mps }
spec:
  selectors:
  - cel:
      expression: >-
        device.attributes["gpu.deckhouse.io"]
          .model == "H100"
```

ResourceClaimTemplate – заявка на долю + бюджет карты

```
kind: ResourceClaimTemplate
spec: { spec: { devices: { requests:
  - name: gpu
    exactly:
      deviceClassName: nvidia-h100-mps
      capacity: { requests: { sharePercent: "25" } }
  } } } } # 25 + 25 + 50 = 100 → пятый под не влезет
```


Pod – связывается с заявкой

kind: Pod

spec:

resourceClaims:

- name: gpu

resourceClaimTemplateName: h100-mps

containers:

- resources: { claims: [{ name: gpu }] }

DRA — это контракт. Движок — на Go

Kubernetes сказал «что».
«Как» — код драйвера на узле.

Планировщик лишь пишет в ResourceClaim:
«этому поду — это устройство».

Создать MIG, поднять MPS, прокинуть карту в контейнер это
не делает ни API-сервер, ни планировщик.

Нужен исполнитель. Это драйвер.

Почему свой драйвер, а не vendor reference

- **У NVIDIA DRA-драйвер уже есть.**
Логично спросить — почему не взять его?

Почему свой драйвер, а не *vendor reference*

- **У NVIDIA DRA-драйвер уже есть.**
Логично спросить — почему не взять его?
- **Dynamic MIG в нём пока нет.** Issue #361, roadmap — MIG нарезается статически

Почему свой драйвер, а не *vendor reference*

- **У NVIDIA DRA-драйвер уже есть.**
Логично спросить — почему не взять его?
- **Dynamic MIG в нём пока нет.** Issue #361, roadmap — MIG нарезается статически
- **MIG и MPS взаимоисключаются.**
А MPS вообще требует целую карту

Почему свой драйвер, а не vendor reference

- **У NVIDIA DRA-драйвер уже есть.**
Логично спросить — почему не взять его?
- **Dynamic MIG в нём пока нет.** Issue #361, roadmap — MIG нарезается статически
- **MIG и MPS взаимоисключаются.**
А MPS вообще требует целую карту
- **Один вендор, один способ.**
Нам нужна одна модель: full / доли / MIG / MPS / разные вендоры

Почему свой драйвер, а не vendor reference

- **У NVIDIA DRA-драйвер уже есть.**
Логично спросить — почему не взять его?
- **Dynamic MIG в нём пока нет.** Issue #361, roadmap — MIG нарезается статически
- **MIG и MPS взаимоисключаются.**
А MPS вообще требует целую карту
- **Один вендор, один способ.**
Нам нужна одна модель: full / доли / MIG / MPS / разные вендоры
- **Поэтому пишем свой.** Железо берём любое, софт — сами, без вендор-лока

Драйвер = Go-сервис с двумя ролями

1. **Публикует инвентарь**
Ведёт ResourceSlice в API: какие карты, в каких режимах, какая capacity
2. **Обслуживает узел**
kubelet зовёт ровно два метода:
Prepare при старте пода
и Unprepare при завершении

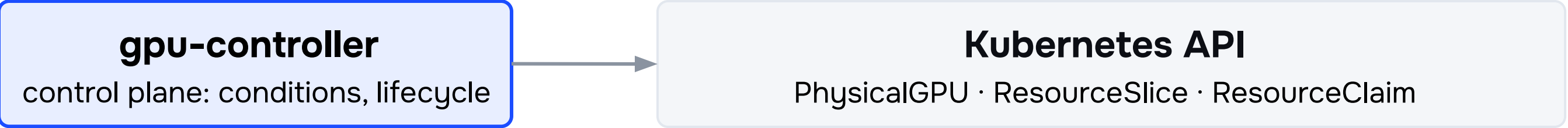
Go-экосистема:

`k8s.io/dynamic-resource-allocation`

`gRPC` поверх `Unix-сокета` · `CDI` (инъекция устройства)

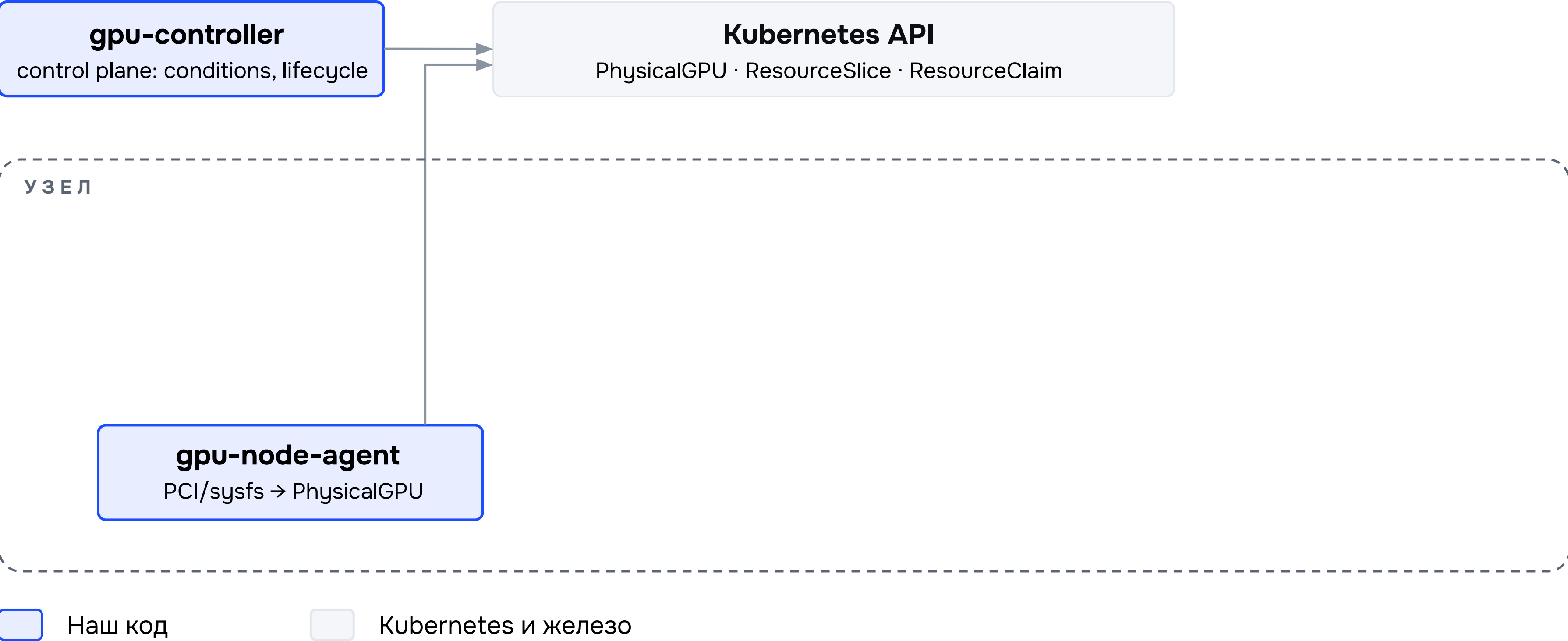
`go-nvml` — `cgo`-биндинги к драйверу `NVIDIA`

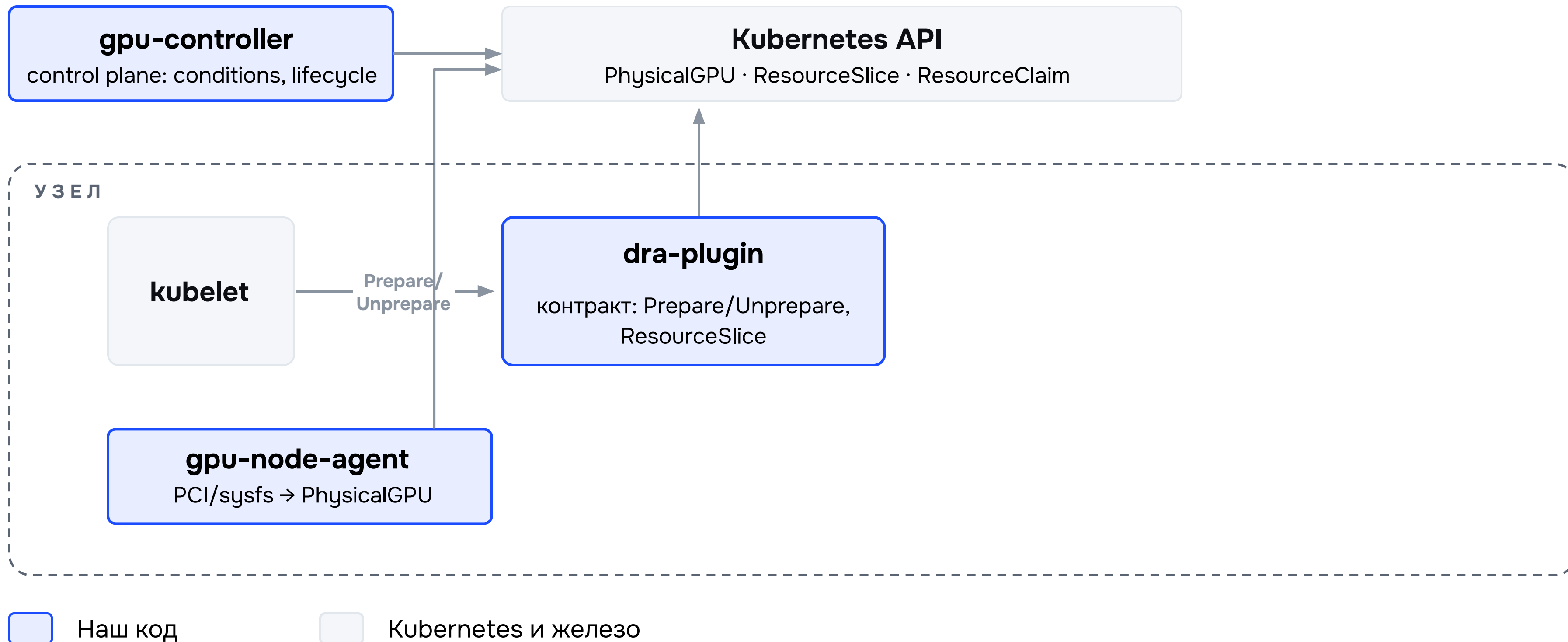
Архитектура: кто с кем говорит

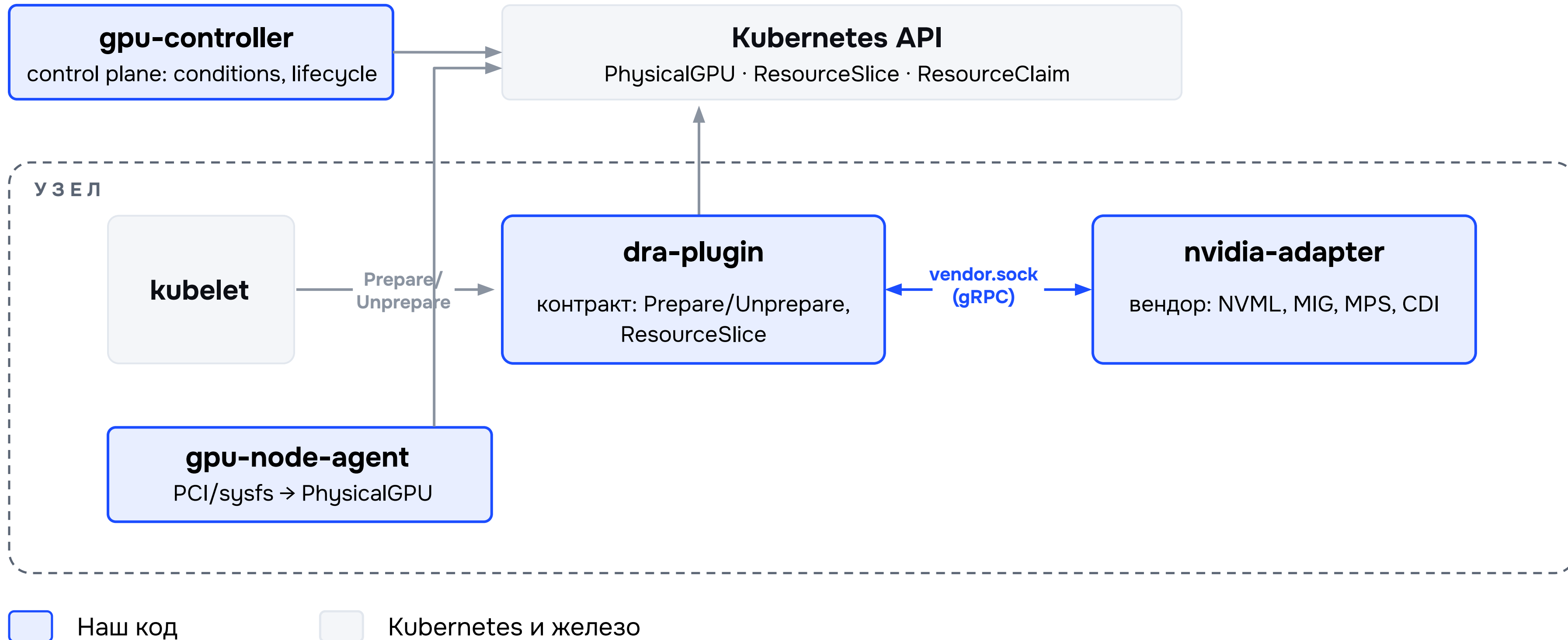


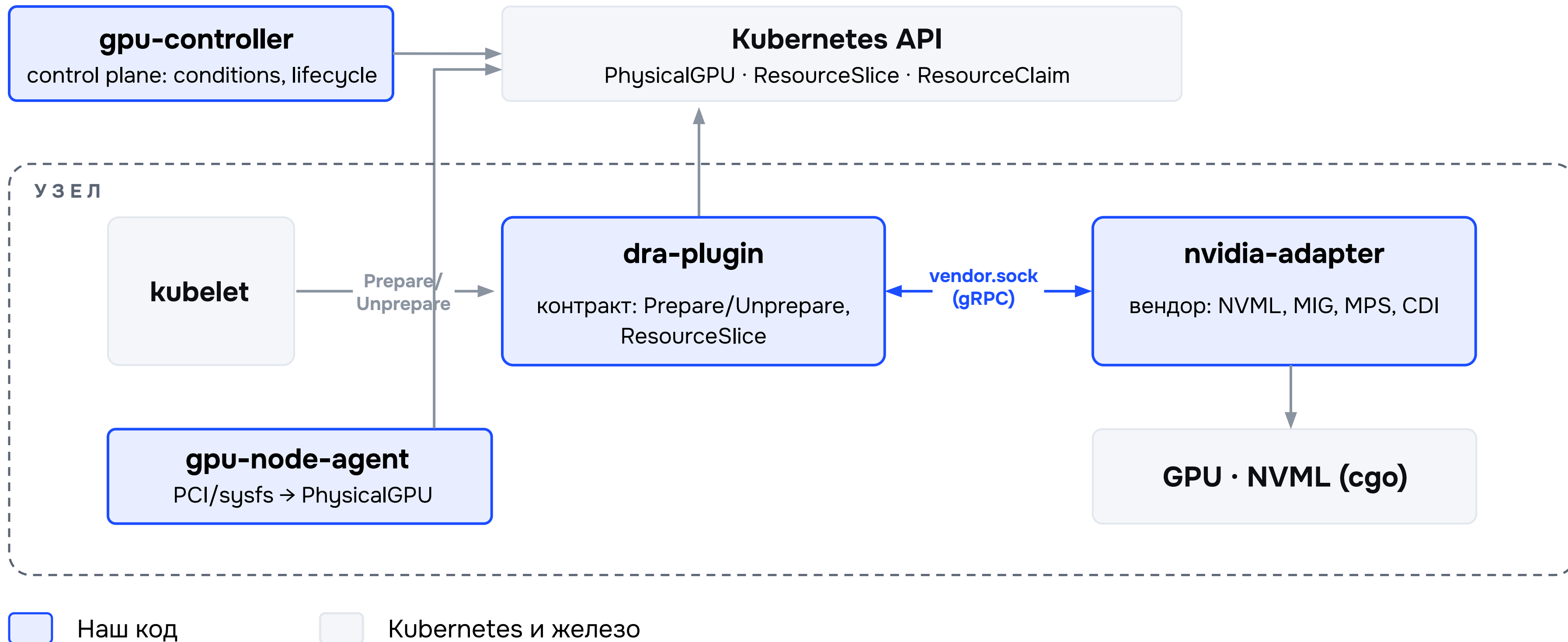
☐ Наш код

☐ Kubernetes и железо









gpu-node-agent

DaemonSet

gpu-node-agent

DaemonSet:

- Сканирует железо узла

gpu-node-agent

DaemonSet:

- Сканирует железо узла
- Читает host sysfs / PCI, находит карты

gpu-node-agent

DaemonSet:

- Сканирует железо узла
- Читает host sysfs / PCI, находит карты

Создаёт и держит актуальным объект PhysicalGPU

ResourceSlice – инвентарь (публикует драйвер)

```
// периодический реконсайд узла
func (r *PhysicalGPUReconciler) reconcile(ctx) (ctrl.Result, error) {
    whitelist, err := r.loadWhitelist(ctx)           // разрешённые вендоры
    vendor, devices, err := r.scanAndSelectDevices(ctx, whitelist) // PCI/sysfs
    r.syncNodeVendorLabel(ctx, vendor)
    r.syncPhysicalGPUObjects(ctx, devices, whitelist) // create / SSA-patch
    return ctrl.Result{RequeueAfter: r.ResyncPeriod}, nil // resync = safety net
}
```

ResourceSlice – инвентарь (публикует драйвер)

```
// периодический реконсайд узла
func (r *PhysicalGPUReconciler) reconcile(ctx) (ctrl.Result, error) {
    whitelist, err := r.loadWhitelist(ctx)           // разрешённые вендоры
    vendor, devices, err := r.scanAndSelectDevices(ctx, whitelist) // PCI/sysfs
    r.syncNodeVendorLabel(ctx, vendor)
    r.syncPhysicalGPUObjects(ctx, devices, whitelist) // create / SSA-patch
    return ctrl.Result{RequeueAfter: r.ResyncPeriod}, nil // resync = safety net
}
```

ResourceSlice – инвентарь (публикует драйвер)

```
// периодический реконсайд узла
func (r *PhysicalGPUReconciler) reconcile(ctx) (ctrl.Result, error) {
    whitelist, err := r.loadWhitelist(ctx)           // разрешённые вендоры
    vendor, devices, err := r.scanAndSelectDevices(ctx, whitelist) // PCI/sysfs
    r.syncNodeVendorLabel(ctx, vendor)
    r.syncPhysicalGPUObjects(ctx, devices, whitelist) // create / SSA-patch
    return ctrl.Result{RequeueAfter: r.ResyncPeriod}, nil // resync = safety net
}
```

ResourceSlice – инвентарь (публикует драйвер)

```
// периодический реконсайд узла
func (r *PhysicalGPUReconciler) reconcile(ctx) (ctrl.Result, error) {
    whitelist, err := r.loadWhitelist(ctx)           // разрешённые вендоры
    vendor, devices, err := r.scanAndSelectDevices(ctx, whitelist) // PCI/sysfs
    r.syncNodeVendorLabel(ctx, vendor)
    r.syncPhysicalGPUObjects(ctx, devices, whitelist) // create / SSA-patch
    return ctrl.Result{RequeueAfter: r.ResyncPeriod}, nil // resync = safety net
}
```

dra-plugin

Vendor-agnostic-фронтенд

dra-plugin

Vendor-agnostic-фронтенд:

- Регистрируется у kubelet, публикует ResourceSlice

dra-plugin

Vendor-agnostic-фронтенд:

- Регистрируется у kubelet, публикует ResourceSlice
- Принимает Prepare/Unprepare

dra-plugin

Vendor-agnostic-фронтенд:

- Регистрируется у kubelet, публикует ResourceSlice
- Принимает Prepare/Unprepare

Ничего не знает про NVIDIA. Говорит только на языке контракта

dra-plugin – регистрация у kubelet + ResourceSlice

```
// 1) регистрируемся у kubelet как DRA-плагин
helper, _ := kubeletplugin.Start(ctx, handler,
    kubeletplugin.DriverName("gpu.deckhouse.io"),
    kubeletplugin.NodeName(cfg.NodeName),
    kubeletplugin.RegistrarDirectoryPath(cfg.RegistrarDir),
    kubeletplugin.PluginDataDirectoryPath(cfg.PluginDataDir))

// 2) публикуем инвентарь: PhysicalGPU → офферы → ResourceSlice
resources := builder.Build(ctx, nodeName, gpus)
if sameHash(resources) { return }
helper.PublishResources(ctx, resources)
```

dra-plugin – регистрация у kubelet + ResourceSlice

```
// 1) регистрируемся у kubelet как DRA-плагин
helper, _ := kubeletplugin.Start(ctx, handler,
    kubeletplugin.DriverName("gpu.deckhouse.io"),
    kubeletplugin.NodeName(cfg.NodeName),
    kubeletplugin.RegistrarDirectoryPath(cfg.RegistrarDir),
    kubeletplugin.PluginDataDirectoryPath(cfg.PluginDataDir))

// 2) публикуем инвентарь: PhysicalGPU → офферы → ResourceSlice
resources := builder.Build(ctx, nodeName, gpus)
if sameHash(resources) { return }
helper.PublishResources(ctx, resources)
```

dra-plugin – регистрация у kubelet + ResourceSlice

```
// 1) регистрируемся у kubelet как DRA-плагин
helper, _ := kubeletplugin.Start(ctx, handler,
    kubeletplugin.DriverName("gpu.deckhouse.io"),
    kubeletplugin.NodeName(cfg.NodeName),
    kubeletplugin.RegistrarDirectoryPath(cfg.RegistrarDir),
    kubeletplugin.PluginDataDirectoryPath(cfg.PluginDataDir))

// 2) публикуем инвентарь: PhysicalGPU → офферы → ResourceSlice
resources := builder.Build(ctx, nodeName, gpus)
if sameHash(resources) { return }
helper.PublishResources(ctx, resources)
```

dra-plugin – регистрация у kubelet + ResourceSlice

```
// 1) регистрируемся у kubelet как DRA-плагин
helper, _ := kubeletplugin.Start(ctx, handler,
    kubeletplugin.DriverName("gpu.deckhouse.io"),
    kubeletplugin.NodeName(cfg.NodeName),
    kubeletplugin.RegistrarDirectoryPath(cfg.RegistrarDir),
    kubeletplugin.PluginDataDirectoryPath(cfg.PluginDataDir))

// 2) публикуем инвентарь: PhysicalGPU → офферы → ResourceSlice
resources := builder.Build(ctx, nodeName, gpus)
if sameHash(resources) { return }
helper.PublishResources(ctx, resources)
```

dra-plugin – регистрация у kubelet + ResourceSlice

```
// 1) регистрируемся у kubelet как DRA-плагин
helper, _ := kubeletplugin.Start(ctx, handler,
    kubeletplugin.DriverName("gpu.deckhouse.io"),
    kubeletplugin.NodeName(cfg.NodeName),
    kubeletplugin.RegistrarDirectoryPath(cfg.RegistrarDir),
    kubeletplugin.PluginDataDirectoryPath(cfg.PluginDataDir))

// 2) публикуем инвентарь: PhysicalGPU → офферы → ResourceSlice
resources := builder.Build(ctx, nodeName, gpus)
if sameHash(resources) { return }
helper.PublishResources(ctx, resources)
```


dra-plugin – регистрация у kubelet + ResourceSlice

```
// 1) регистрируемся у kubelet как DRA-плагин
helper, _ := kubeletplugin.Start(ctx, handler,
    kubeletplugin.DriverName("gpu.deckhouse.io"),
    kubeletplugin.NodeName(cfg.NodeName),
    kubeletplugin.RegistrarDirectoryPath(cfg.RegistrarDir),
    kubeletplugin.PluginDataDirectoryPath(cfg.PluginDataDir))

// 2) публикуем инвентарь: PhysicalGPU → офферы → ResourceSlice
resources := builder.Build(ctx, nodeName, gpus)
if sameHash(resources) { return }
helper.PublishResources(ctx, resources)
```

dra-plugin – где живёт тяжёлая работа

```
// kubelet зовёт по plugin-сокету
func (h *Handler) PrepareResourceClaims(ctx,
    claims []*resourcev1.ResourceClaim,
) (map[types.UID]kubeletplugin.PrepareResult, error) {
    for _, claim := range claims {
        if r, ok := h.cache.Get(claim.UID); ok { // идемпотентность
            results[uid] = r; continue }
        devices, _ := h.resolveClaimDevices(ctx, claim) // что выделил scheduler
        prepared, _ := h.prepareCDIDevices(ctx, claim, devices) // gRPC →
        vendor.sock
        results[uid] = PrepareResult{Devices: prepared}
    } }
```

dra-plugin – где живёт тяжёлая работа

```
// kubelet зовёт по plugin-сокету
func (h *Handler) PrepareResourceClaims(ctx,
    claims []*resourcev1.ResourceClaim,
) (map[types.UID]kubeletplugin.PrepareResult, error) {
    for _, claim := range claims {
        if r, ok := h.cache.Get(claim.UID); ok { // идемпотентность
            results[uid] = r; continue }
        devices, _ := h.resolveClaimDevices(ctx, claim) // что выделил scheduler
        prepared, _ := h.prepareCDIDevices(ctx, claim, devices) // gRPC →
        vendor.sock
        results[uid] = PrepareResult{Devices: prepared}
    } }
```


dra-plugin – где живёт тяжёлая работа

```
// kubelet зовёт по plugin-сокету
func (h *Handler) PrepareResourceClaims(ctx,
    claims []*resourcev1.ResourceClaim,
) (map[types.UID]kubeletplugin.PrepareResult, error) {
    for _, claim := range claims {
        if r, ok := h.cache.Get(claim.UID); ok { // идемпотентность
            results[uid] = r; continue }
        devices, _ := h.resolveClaimDevices(ctx, claim) // что выделил scheduler
        prepared, _ := h.prepareCDIDevices(ctx, claim, devices) // gRPC →
        vendor.sock
        results[uid] = PrepareResult{Devices: prepared}
    } }
```

dra-plugin – где живёт тяжёлая работа

```
// kubelet зовёт по plugin-сокету
func (h *Handler) PrepareResourceClaims(ctx,
    claims []*resourcev1.ResourceClaim,
) (map[types.UID]kubeletplugin.PrepareResult, error) {
    for _, claim := range claims {
        if r, ok := h.cache.Get(claim.UID); ok { // идемпотентность
            results[uid] = r; continue }
        devices, _ := h.resolveClaimDevices(ctx, claim) // что выделил scheduler
        prepared, _ := h.prepareCDIDevices(ctx, claim, devices) // gRPC →
vendor.sock
        results[uid] = PrepareResult{Devices: prepared}
    } }
```

dra-plugin – где живёт тяжёлая работа

```
// kubelet зовёт по plugin-сокету
func (h *Handler) PrepareResourceClaims(ctx,
    claims []*resourcev1.ResourceClaim,
) (map[types.UID]kubeletplugin.PrepareResult, error) {
    for _, claim := range claims {
        if r, ok := h.cache.Get(claim.UID); ok { // идемпотентность
            results[uid] = r; continue }
        devices, _ := h.resolveClaimDevices(ctx, claim) // что выделил scheduler
        prepared, _ := h.prepareCDIDevices(ctx, claim, devices) // gRPC →
        vendor.sock
        results[uid] = PrepareResult{Devices: prepared}
    } }
```

nvidia-adapter

Vendor-backend: вся работа с железом

nvidia-adapter

Vendor-backend: вся работа с железом

- NVML через cgo, dynamic MIG, MPS

nvidia-adapter

Vendor-backend: вся работа с железом

- NVML через sgo, dynamic MIG, MPS
- Генерация CDI

nvidia-adapter

Vendor-backend: вся работа с железом

- NVML через cgo, dynamic MIG, MPS
- Генерация CDI

auto-MIG planner и **MPS manager** — внутренние пакеты, не отдельные бинари

nvidia-adapter – Prepare как конвейер шагов

```
func (e *Engine) prepareSteps(st *PrepareState) Steps {  
    return Steps{  
        {Name: "lock"},           // клейм-лок  
        {Name: "checkpointLoad"}, // пережить рестарт kubelet  
        {Name: "plan"},           // что создать/удалить  
        {Name: "autoMigPlan"},     // детерминированный auto-MIG  
        {Name: "maintenanceEnter"}, // Reconfigure: RequireGPUPFree  
        {Name: "gpuFreeCheck"},    // ждём, пока карта свободна  
        {Name: "migModeApply"},    // full ⇨ MIG на лету  
        {Name: "migPrepare"},      // создать GI/CI (go-nvml)  
        {Name: "sharingPrepare"},  // MPS / time-slicing  
    }  
}
```


nvidia-adapter – Prepare как конвейер шагов

```
func (e *Engine) prepareSteps(st *PrepareState) Steps {  
    return Steps{  
        {Name: "lock"},           // клейм-лок  
        {Name: "checkpointLoad"}, // пережить рестарт kubelet  
        {Name: "plan"},           // что создать/удалить  
        {Name: "autoMigPlan"},     // детерминированный auto-MIG  
        {Name: "maintenanceEnter"}, // Reconfigure: RequireGPUPFree  
        {Name: "gpuFreeCheck"},    // ждём, пока карта свободна  
        {Name: "migModeApply"},    // full ⇐ MIG на лету  
        {Name: "migPrepare"},      // создать GI/CI (go-nvml)  
        {Name: "sharingPrepare"},  // MPS / time-slicing  
    }  
}
```

nvidia-adapter – Prepare как конвейер шагов

```
func (e *Engine) prepareSteps(st *PrepareState) Steps {  
    return Steps{  
        {Name: "lock"},           // клейм-лок  
        {Name: "checkpointLoad"}, // пережить рестарт kubelet  
        {Name: "plan"},           // что создать/удалить  
        {Name: "autoMigPlan"},    // детерминированный auto-MIG  
        {Name: "maintenanceEnter"}, // Reconfigure: RequireGPUPFree  
        {Name: "gpuFreeCheck"},    // ждём, пока карта свободна  
        {Name: "migModeApply"},    // full  $\Rightarrow$  MIG на лету  
        {Name: "migPrepare"},      // создать GI/CI (go-nvml)  
        {Name: "sharingPrepare"},  // MPS / time-slicing  
    }  
}
```

nvidia-adapter – Prepare как конвейер шагов

```
func (e *Engine) prepareSteps(st *PrepareState) Steps {  
    return Steps{  
        {Name: "lock"},           // клейм-лок  
        {Name: "checkpointLoad"}, // пережить рестарт kubelet  
        {Name: "plan"},           // что создать/удалить  
        {Name: "autoMigPlan"},    // детерминированный auto-MIG  
        {Name: "maintenanceEnter"}, // Reconfigure: RequireGPUPFree  
        {Name: "gpuFreeCheck"},    // ждём, пока карта свободна  
        {Name: "migModeApply"},    // full ≠ MIG на лету  
        {Name: "migPrepare"},      // создать GI/CI (go-nvml)  
        {Name: "sharingPrepare"},  // MPS / time-slicing  
    }  
}
```

nvidia-adapter – Prepare как конвейер шагов

```
func (e *Engine) prepareSteps(st *PrepareState) Steps {  
    return Steps{  
        {Name: "lock"},           // клейм-лок  
        {Name: "checkpointLoad"}, // пережить рестарт kubelet  
        {Name: "plan"},           // что создать/удалить  
        {Name: "autoMigPlan"},    // детерминированный auto-MIG  
        {Name: "maintenanceEnter"}, // Reconfigure: RequireGPUPFree  
        {Name: "gpuFreeCheck"},    // ждём, пока карта свободна  
        {Name: "migModeApply"},    // full ⇐ MIG на лету  
        {Name: "migPrepare"},      // создать GI/CI (go-nvml)  
        {Name: "sharingPrepare"},  // MPS / time-slicing  
    }  
}
```

nvidia-adapter – Prepare как конвейер шагов

```
func (e *Engine) prepareSteps(st *PrepareState) Steps {  
    return Steps{  
        {Name: "lock"},           // клейм-лок  
        {Name: "checkpointLoad"}, // пережить рестарт kubelet  
        {Name: "plan"},           // что создать/удалить  
        {Name: "autoMigPlan"},    // детерминированный auto-MIG  
        {Name: "maintenanceEnter"}, // Reconfigure: RequireGPUPFree  
        {Name: "gpuFreeCheck"},    // ждём, пока карта свободна  
        {Name: "migModeApply"},    // full ⇐ MIG на лету  
        {Name: "migPrepare"},      // создать GI/CI (go-nvml)  
        {Name: "sharingPrepare"},  // MPS / time-slicing  
    }  
}
```


gpu-controller

Control plane

gpu-controller

Control plane:

- Реконсайт PhysicalGPU и DRA-объекты в масштабе кластера

gpu-controller

Control plane:

- Реконсайт PhysicalGPU и DRA-объекты в масштабе кластера
- Conditions, server-side apply

gpu-controller – реконсайд цепочкой хендлеров

```
func (r *Reconciler) buildHandlerChain() []Handler {  
    return []Handler{  
        handlers.NewDiscoverHandler(r.driverName),    // узел, активные claims  
        handlers.NewDeletionHandler(),  
        handlers.NewCapabilitiesHandler(),            // что карта умеет  
        handlers.NewPowerLimitHandler(),  
        handlers.NewProtectionHandler(),              // finalizer  
        handlers.NewLifecycleHandler(...),            // phase + conditions (SSA)  
        handlers.NewFilterReadyHandler(),  
        handlers.NewFilterHealthyHandler(),  
    }  
}
```

gpu-controller – реконсайд цепочкой хендлеров

```
func (r *Reconciler) buildHandlerChain() []Handler {  
    return []Handler{  
        handlers.NewDiscoverHandler(r.driverName),    // узел, активные claims  
        handlers.NewDeletionHandler(),  
        handlers.NewCapabilitiesHandler(),            // что карта умеет  
        handlers.NewPowerLimitHandler(),  
        handlers.NewProtectionHandler(),              // finalizer  
        handlers.NewLifecycleHandler(...),            // phase + conditions (SSA)  
        handlers.NewFilterReadyHandler(),  
        handlers.NewFilterHealthyHandler(),  
    }  
}
```

gpu-controller – реконсайд цепочкой хендлеров

```
func (r *Reconciler) buildHandlerChain() []Handler {  
    return []Handler{  
        handlers.NewDiscoverHandler(r.driverName),    // узел, активные claims  
        handlers.NewDeletionHandler(),  
        handlers.NewCapabilitiesHandler(),             // что карта умеет  
        handlers.NewPowerLimitHandler(),  
        handlers.NewProtectionHandler(),              // finalizer  
        handlers.NewLifecycleHandler(...),            // phase + conditions (SSA)  
        handlers.NewFilterReadyHandler(),  
        handlers.NewFilterHealthyHandler(),  
    }  
}
```

gpu-controller – реконсайд цепочкой хендлеров

```
func (r *Reconciler) buildHandlerChain() []Handler {  
    return []Handler{  
        handlers.NewDiscoverHandler(r.driverName),    // узел, активные claims  
        handlers.NewDeletionHandler(),  
        handlers.NewCapabilitiesHandler(),            // что карта умеет  
        handlers.NewPowerLimitHandler(),  
        handlers.NewProtectionHandler(),              // finalizer  
        handlers.NewLifecycleHandler(...),            // phase + conditions (SSA)  
        handlers.NewFilterReadyHandler(),  
        handlers.NewFilterHealthyHandler(),  
    }  
}
```

gpu-controller – реконсайт цепочкой хендлеров

```
func (r *Reconciler) buildHandlerChain() []Handler {  
    return []Handler{  
        handlers.NewDiscoverHandler(r.driverName),    // узел, активные claims  
        handlers.NewDeletionHandler(),  
        handlers.NewCapabilitiesHandler(),            // что карта умеет  
        handlers.NewPowerLimitHandler(),  
        handlers.NewProtectionHandler(),              // finalizer  
        handlers.NewLifecycleHandler(...),              // phase + conditions (SSA)  
        handlers.NewFilterReadyHandler(),  
        handlers.NewFilterHealthyHandler(),  
    }  
}
```


gpu-controller – реконсайт цепочкой хендлеров

```
func (r *Reconciler) buildHandlerChain() []Handler {  
    return []Handler{  
        handlers.NewDiscoverHandler(r.driverName),    // узел, активные claims  
        handlers.NewDeletionHandler(),  
        handlers.NewCapabilitiesHandler(),            // что карта умеет  
        handlers.NewPowerLimitHandler(),  
        handlers.NewProtectionHandler(),              // finalizer  
        handlers.NewLifecycleHandler(...),            // phase + conditions (SSA)  
        handlers.NewFilterReadyHandler(),  
        handlers.NewFilterHealthyHandler(),  
    }  
}
```

**Prepare/Unprepare –
место всей логики**

Prepare/Unprepare – место всей логики

- Создание MIG-инстансов под конкретный запрос

Prepare/Unprepare – место всей логики

- Создание MIG-инстансов под конкретный запрос
- Запуск MPS с active thread % и pinned memory limit

Prepare/Unprepare – место всей логики

- Создание MIG-инстансов под конкретный запрос
- Запуск MPS с active thread % и pinned memory limit
- CDI-инъекция: как устройство попадает в контейнер

Device plugin научил Kubernetes видеть GPU.
DRA дал нормальную модель устройства.
Драйвер на Go даёт реализацию.

*Утилизация дорогого железа = деньги,
и теперь это управляемо.*

Спасибо за внимание!

 [deckhouse_news](#)

 github.com/deckhouse



статья Habr

Backup

Код для вопросов:
SSA-инвентарь и NVML-нарезка MIG

gpu-node-agent – PhysicalGPU

через server-side apply

```
patch := nodeAgentStatusApply{
  TypeMeta:    metav1.TypeMeta{Kind: "PhysicalGPU", APIVersion: gv},
  ObjectMeta:  metav1.ObjectMeta{Name: name},
  Status: nodeAgentStatusFields{ // владеем ТОЛЬКО этим
    NodeInfo: nodeInfo, PCIInfo: pciInfo,
  },
}
data, _ := json.Marshal(patch)

// ForceOwnership HE ставим: status.capabilities – за vendor-handler
return c.Status().Patch(ctx, obj,
  client.RawPatch(types.ApplyPatchType, data),
  client.FieldOwner(gpuv1alpha1.FieldManagerNodeAgent))
```

gpu-node-agent – PhysicalGPU

через server-side apply

```
patch := nodeAgentStatusApply{
  TypeMeta:    metav1.TypeMeta{Kind: "PhysicalGPU", APIVersion: gv},
  ObjectMeta:  metav1.ObjectMeta{Name: name},
  Status: nodeAgentStatusFields{          // владеем ТОЛЬКО этим
    NodeInfo: nodeInfo, PCIInfo: pciInfo,
  },
}
data, _ := json.Marshal(patch)

// ForceOwnership НЕ ставим: status.capabilities – за vendor-handler
return c.Status().Patch(ctx, obj,
  client.RawPatch(types.ApplyPatchType, data),
  client.FieldOwner(gpuv1alpha1.FieldManagerNodeAgent))
```

gpu-node-agent – PhysicalGPU

через server-side apply

```
patch := nodeAgentStatusApply{
  TypeMeta:    metav1.TypeMeta{Kind: "PhysicalGPU", APIVersion: gv},
  ObjectMeta:  metav1.ObjectMeta{Name: name},
  Status: nodeAgentStatusFields{           // владеем ТОЛЬКО этим
    NodeInfo: nodeInfo, PCIInfo: pciInfo,
  },
}
data, _ := json.Marshal(patch)

// ForceOwnership HE ставим: status.capabilities – за vendor-handler
return c.Status().Patch(ctx, obj,
  client.RawPatch(types.ApplyPatchType, data),
  client.FieldOwner(gpuv1alpha1.FieldManagerNodeAgent))
```


gpu-node-agent – PhysicalGPU через server-side apply

```
patch := nodeAgentStatusApply{
    TypeMeta:    metav1.TypeMeta{Kind: "PhysicalGPU", APIVersion: gv},
    ObjectMeta:  metav1.ObjectMeta{Name: name},
    Status: nodeAgentStatusFields{           // владеем ТОЛЬКО этим
        NodeInfo: nodeInfo, PCIInfo: pciInfo,
    },
}
data, _ := json.Marshal(patch)

// ForceOwnership HE ставим: status.capabilities – за vendor-handler
return c.Status().Patch(ctx, obj,
    client.RawPatch(types.ApplyPatchType, data),
    client.FieldOwner(gpuv1alpha1.FieldManagerNodeAgent)))
```

nvidia-adapter – нарезка MIG через NVML (cgo)

```
// GPU Instance → Compute Instance, go-nvml v0.13
gi, ret := device.CreateGpuInstanceWithPlacement(&giInfo, &giPlacement)
if ret == nvml.ERROR_INSUFFICIENT_RESOURCES {
    gi, ok = findExistingGPUInstance(device, giInfo, req) // переиспользуем
}

ci, ret := gi.CreateComputeInstance(&ciInfo) // внутри GI

// Unprepare – симметрично, в обратном порядке:
_ = lib.ComputeInstanceDestroy(ci)
_ = lib.GpuInstanceDestroy(gi)
```

nvidia-adapter – нарезка MIG через NVML (cgo)

```
// GPU Instance → Compute Instance, go-nvml v0.13
gi, ret := device.CreateGpuInstanceWithPlacement(&giInfo, &giPlacement)
if ret == nvml.ERROR_INSUFFICIENT_RESOURCES {
    gi, ok = findExistingGPUInstance(device, giInfo, req) // переиспользуем
}

ci, ret := gi.CreateComputeInstance(&ciInfo) // внутри GI

// Unprepare – симметрично, в обратном порядке:
_ = lib.ComputeInstanceDestroy(ci)
_ = lib.GpuInstanceDestroy(gi)
```

nvidia-adapter – нарезка MIG через NVML (cgo)

```
// GPU Instance → Compute Instance, go-nvml v0.13
gi, ret := device.CreateGpuInstanceWithPlacement(&giInfo, &giPlacement)
if ret == nvml.ERROR_INSUFFICIENT_RESOURCES {
    gi, ok = findExistingGPUInstance(device, giInfo, req) // переиспользуем
}

ci, ret := gi.CreateComputeInstance(&ciInfo) // внутри GI

// Unprepare – симметрично, в обратном порядке:
_ = lib.ComputeInstanceDestroy(ci)
_ = lib.GpuInstanceDestroy(gi)
```

nvidia-adapter – нарезка MIG через NVML (cgo)

```
// GPU Instance → Compute Instance, go-nvml v0.13
gi, ret := device.CreateGpuInstanceWithPlacement(&giInfo, &giPlacement)
if ret == nvml.ERROR_INSUFFICIENT_RESOURCES {
    gi, ok = findExistingGPUInstance(device, giInfo, req) // переиспользуем
}

ci, ret := gi.CreateComputeInstance(&ciInfo) // внутри GI

// Unprepare – симметрично, в обратном порядке:
_ = lib.ComputeInstanceDestroy(ci)
_ = lib.GpuInstanceDestroy(gi)
```